LCA

人员

于子珈、杨瑾硕、周子一、陈洛冉、谢亚锴、司云心、赵熙羽 到课, 李子瀚 线上

上周作业检查

上周作业链接: https://cppoj.kids123code.com/contest/3194

刷新成功

王向东老师周日十点半C++ST表

刷新

#	用户名	姓名	编程分	时间	A	B	C	D
1	yangjinshuo	杨谨硕	300	2213	100	100	100	
2	yuzijia1	于子珈	300	2439	100	100	100	
3	chenluoran	陈洛冉	300	3991	100	100	100	
4	zhouziyi	周子一	300	6209	100	100	100	
5	niutongze	牛同泽	200	2036	100	100		
6	siyunxin	司云心	200	2109	100	100		
7	zhaoxiyu	赵熙羽	200	2199	100	100		
8	xieyakai	谢亚锴	200	2374	100	100	0	
9	lizihan	李子瀚	200	2530	100	100		

本周作业

https://cppoj.kids123code.com/contest/3300 (课上讲了 A ~ C 题, 课后作业是 B C 题)

课堂表现

今天讲了 LCA 这个知识点, 同学们课上整体理解的都比较好, 建议同学们课下要把 LCA 的模板练熟。

课堂内容

【模板】最近公共祖先（LCA）

倍增法求 最近公共祖先

f[i][j]: i 这个点, 往上 2^j 的父亲是谁

(a,b) 的最近公共祖先: 先倍增往上跳, 直到两个点变到一层; 然后倍增往上判, 只要两个点不重合就往上, 最后返回 2 个点的父结点

```
#include <bits/stdc++.h>

using namespace std;

const int N = 5e5 + 5, M = 20;
vector<int> vec[N];
```

```cpp
int dis[N], f[N][M];

void dfs(int u, int fa, int d) {
  dis[u] = d; f[u][0] = fa;
  for (int i = 1; i < M; ++i) f[u][i] = f[f[u][i-1]][i-1];
  for (int i : vec[u]) {
    if (i == fa) continue;
    dfs(i, u, d+1);
  }
}

int lca(int a, int b) {
  if (dis[a] < dis[b]) return lca(b, a);

  for (int i = M-1; i >= 0; --i) {
    if (dis[f[a][i]] >= dis[b]) a = f[a][i];
  }

  if (a == b) return a;

  for (int i = M-1; i >= 0; --i) {
    if (f[a][i] != f[b][i]) a = f[a][i], b = f[b][i];
  }
  return f[a][0];
}

int main()
{
  ios::sync_with_stdio(false);
  cin.tie(0);

  int n, m, root; cin >> n >> m >> root;
  for (int i = 1; i <= n-1; ++i) {
    int a, b; cin >> a >> b;
    vec[a].push_back(b), vec[b].push_back(a);
  }

  dfs(root, 0, 1);

  while (m -- ) {
    int a, b; cin >> a >> b;
    cout << lca(a, b) << "\n";
  }
  return 0;
}
```

[GESP202312 八级] 大量的工作沟通

dfs 中, 维护 ans[i]: 从根到 i 中最大的编号是多少

找到所有点的 LCA 点 id, 答案即为 ans[id]

```cpp
#include <bits/stdc++.h>

using namespace std;

const int N = 1e5 + 5, M = 20;
vector<int> vec[N];
int dis[N], f[N][M], ans[N];

void dfs(int u, int fa, int d) {
  dis[u] = d; f[u][0] = fa; ans[u] = max(ans[fa], u);
  for (int i = 1; i < M; ++i) f[u][i] = f[f[u][i-1]][i-1];
  for (int i : vec[u]) {
    if (i == fa) continue;
    dfs(i, u, d+1);
  }
}

int lca(int a, int b) {
  if (dis[a] < dis[b]) return lca(b, a);

  for (int i = M-1; i >= 0; --i) {
    if (dis[f[a][i]] >= dis[b]) a = f[a][i];
  }

  if (a == b) return a;

  for (int i = M-1; i >= 0; --i) {
    if (f[a][i] != f[b][i]) a = f[a][i], b = f[b][i];
  }
  return f[a][0];
}

void solve() {
  int m; cin >> m;
  int fa; cin >> fa; ++fa;
  for (int i = 2; i <= m; ++i) {
    int x; cin >> x; ++x; fa = lca(fa, x);
  }

  cout << ans[fa] - 1 << endl;
}

int main()
{
  int n; cin >> n;
  for (int i = 2; i <= n; ++i) {
    int id; cin >> id; ++id; vec[id].push_back(i);
  }

  dfs(1, 0, 1);

  int T; cin >> T;
  while (T -- ) solve();
```

```
    return 0;
  }
```

[蓝桥杯 2025 国 Python A] 巡逻

维护 2 个数组

dis[i]: 从根到 i, 中间的点数

p[i]: 从根到 i, 中间的风险和

树上任意两点 i j 之间的距离为: dis[i] + dis[j] - 2*dis[fa]

树上任意两点 i j 之间的路径和为: p[i] + p[j] - 2*p[fa]

```
#include <bits/stdc++.h>
#define int long long

using namespace std;

const int N = 5000 + 5, M = 20;
struct node {
  int to, value;
};
vector<node> vec[N];
int dis[N], f[N][M], p[N];

void dfs(int u, int fa, int d) {
  dis[u] = d; f[u][0] = fa;
  for (int i = 1; i < M; ++i) f[u][i] = f[f[u][i-1]][i-1];
  for (node it : vec[u]) {
    if (it.to == fa) continue;
    p[it.to] = p[u] + it.value;
    dfs(it.to, u, d+1);
  }
}

int lca(int a, int b) {
  if (dis[a] < dis[b]) return lca(b, a);

  for (int i = M-1; i >= 0; --i) {
    if (dis[f[a][i]] >= dis[b]) a = f[a][i];
  }

  if (a == b) return a;

  for (int i = M-1; i >= 0; --i) {
    if (f[a][i] != f[b][i]) a = f[a][i], b = f[b][i];
  }
  return f[a][0];
}
```

```
signed main()
{
  int n, K; cin >> n >> K;
  for (int i = 1; i <= n-1; ++i) {
    int a, b, c; cin >> a >> b >> c;
    vec[a].push_back({b,c}), vec[b].push_back({a,c});
  }

  dfs(1, 0, 1);

  int res = 0;
  for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= n; ++j) {
      int fa = lca(i, j);
      if (dis[i]+dis[j]-2*dis[fa] == K) res += p[i]+p[j]-2*p[fa];
    }
  }
  cout << res << endl;
  return 0;
}
```